

# 基于CNN的人脸识别

## 一、实验内容

- 手动搭建基于CNN的人脸识别网络
- 利用fetch\_olivetti\_faces人脸数据集进行训练人脸识别模型
- 测试已训练好人脸识别模型性能

## 二、实验目标

- 理解基于CNN的人脸识别网络搭建过程
- 掌握数据获取和处理以及网络训练和测试流程
- 输出人脸识别模型训练和测试结果

## 三、实验环境

- Python 3.7.11
- Keras 2.5.0
- numpy 1.19.5
- matplotlib 3.2.2
- scikit-learn 0.22.2.post1

## 四、实验原理及步骤

### 步骤1：导入相关包

```
from sklearn import datasets
from matplotlib import pyplot as plt
from sklearn.model_selection import train_test_split
import keras
import numpy as np
```

### 步骤2：获取数据集

- 数据集来自sklearn的datasets的fetch\_olivetti\_faces，只有400张，在训练CNN时，占用时间少，方便测试

```
faces = datasets.fetch_olivetti_faces()
```

输出结果

```
downloading Olivetti faces from https://ndownloader.figshare.com/files/5976027 to
/root/scikit_learn_data
```

- 通过faces.images就是人脸对应的图像数组，我们先看下shape

```
print(faces.images.shape)
```

输出结果

(400, 64, 64)

- 一共是400张64×64的图片

### 步骤3：数据可视化

```
i = 0
for img in faces.images:
    #总共400张图
    plt.imshow(img, cmap="gray")
    #关闭x, y轴显示
    plt.xticks([])
    plt.yticks([])
    plt.xlabel(faces.target[i])
    plt.show()
    i = i + 1
```

输出结果

Output hidden; open in <https://colab.research.google.com> to view.

- 从图中可以看到，每个人有10个头像，对应的人脸标签是从0-39，一共40种人脸

### 步骤4：数据处理

- 了解了数据后，我们准备好特征数据和对应标签

```
#人脸数据
x = faces.images
#人脸对应的标签
y = faces.target
print(x[0])
print(y[0])
```

输出结果

```
[[0.30991736 0.3677686 0.41735536 ... 0.37190083 0.3305785 0.30578512]
 [0.3429752 0.40495867 0.43801653 ... 0.37190083 0.338843 0.3140496 ]
 [0.3429752 0.41735536 0.45041323 ... 0.38016528 0.338843 0.29752067]
 ...
 [0.21487603 0.20661157 0.2231405 ... 0.15289256 0.16528925 0.17355372]
 [0.20247933 0.2107438 0.2107438 ... 0.14876033 0.16115703 0.16528925]
 [0.20247933 0.20661157 0.20247933 ... 0.15289256 0.16115703 0.1570248 ]]
0
```

- 每个images对应的是已经归一化的像素数据，每个target对应人脸的标签
- 首先要reshape一下数据格式，由原本的[一次的训练数量，长，宽]，变为[一次的训练数量，长，宽，通道数]，通道数实际上就是深度，我们本次样本是黑白图，所以深度只有1，如果是RGB彩色照片，通道数就是3，这个通道数也可以自己设计

```
x = x.reshape(400, 64, 64, 1)
```

- 随机分割30%的数据做测试的数据

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

## 步骤5：构建CNN人脸识别模型

- Sequential顺序模型是多个网络层的线性堆叠  
通过将网络层实例的列表传递给Sequential的构造器，来创建一个Sequential模型，也可以简单地使用.add()方法将各层添加到模型中
- Conv2D是2D卷积层(例如对图像的空间卷积)。该层创建了一个卷积核，该卷积核对层输入进行卷积，以生成输出张量。如果use\_bias为True，则会创建一个偏置向量并将其添加到输出中。最后，如果activation不是None，它也会应用于输出。当使用该层作为模型第一层时，需要提供input\_shape参数(整数元组，不包含batch轴)，例如，input\_shape=(128, 128, 3)表示128x128 RGB图像，在data\_format="channels\_last"时

**关键参数：**

**filters：**整数，输出空间的维度(即卷积中滤波器的输出数量)

**kernel\_size：**一个整数，或者2个整数表示的元组或列表，指明2D卷积窗口的宽度和高度。可以是一个整数，为所有空间维度指定相同的值

**strides：**一个整数，或者2个整数表示的元组或列表，指明卷积沿宽度和高度方向的步长。可以是一个整数，为所有空间维度指定相同的值。指定任何stride值!=1与指定dilation\_rate值!=1两者不兼容

**padding：**"valid"或"same"(大小写敏感)

**data\_format：**字符串，channels\_last(默认)或channels\_first之一，表示输入中维度的顺序。channels\_last对应输入尺寸为(batch, height, width, channels)，channels\_first对应输入尺寸为(batch, channels, height, width)。它默认为从Keras配置文件~/.keras/keras.json中找到的image\_data\_format值。如果你从未设置它，将使用channels\_last

**dilation\_rate：**一个整数或2个整数的元组或列表，指定膨胀卷积的膨胀率。可以是一个整数，为所有空间维度指定相同的值。当前，指定任何dilation\_rate值!=1与指定stride值!=1两者不兼容

**activation：**要使用的激活函数，如果不指定，则不使用激活函数

**use\_bias：**布尔值，该层是否使用偏置向量

**kernel\_initializer：**kernel权值矩阵的初始化器

**bias\_initializer：**偏置向量的初始化器

**kernel\_regularizer：**运用到kernel权值矩阵的正则化函数

**bias\_regularizer：**运用到偏置向量的正则化函数

**activity\_regularizer：**运用到层输出(它的激活值)的正则化函数

**kernel\_constraint：**运用到kernel权值矩阵的约束函数

**bias\_constraint：**运用到偏置向量的约束函数

- Flatten将输入展平，不影响批量大小

**关键参数：**

**data\_format：**一个字符串，其值为channels\_last(默认值)或者channels\_first。它表明输入的维度的顺序。此参数的目的是当模型从一种数据格式切换到另一种数据格式时保留权重顺序。channels\_last对应着尺寸为(batch, ..., channels)的输入，而channels\_first对应着尺寸为(batch, channels, ...)的输入。默认为image\_data\_format的值，你可以在Keras的配置文件~/.keras/keras.json中找到它。如果你从未设置过它，那么它将是channels\_last

- Dense是全连接层

**关键参数：**

**units：**正整数，输出空间维度

**input\_shape：**现在模型就会以尺寸为(\*, 100)的数组作为输入，其输出数组的尺寸为(\*, 128)，在第一层之后，你就不再需要指定输入的尺寸了

**activation：**激活函数，若不指定，则不使用激活函数

- compile是用于配置训练模型

**关键参数：**

**optimizer：**字符串(优化器名)或者优化器对象

**loss** : 字符串(目标函数名)或目标函数或Loss实例

**metrics** : 在训练和测试期间的模型评估标准, 通常使用metrics=['accuracy']

**loss\_weights** : 指定标量系数(Python浮点数)的可选列表或字典, 用于加权不同模型输出的损失贡献。模型将要最小化的损失值将是所有单个损失的加权和, 由loss\_weights系数加权。如果是列表, 则期望与模型的输出具有1:1映射。如果是字典, 则期望将输出名称(字符串)映射到标量系数

```
model = keras.Sequential()
# 第一层卷积, 卷积的数量为128, 卷积的高和宽是3x3, 激活函数使用relu
model.add(keras.layers.Conv2D(128, kernel_size=3, activation='relu',
input_shape=(64, 64, 1)))
# 第二层卷积
model.add(keras.layers.Conv2D(64, kernel_size=3, activation='relu'))
#把多维数组压缩成一维, 里面的操作可以简单理解为reshape, 方便后面Dense使用
model.add(keras.layers.Flatten())
#对应cnn的全链接层, 可以简单理解为把上面的小图汇集起来, 进行分类
model.add(keras.layers.Dense(40, activation='softmax'))

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

## 步骤6 : 训练

- 训练比较简单, 通过fit就会开始训练, 我们这里定义的是10次
- fit以固定数量的轮次(数据集上的迭代)训练模型

**关键参数 :**

**x** : 输入数据。可以是 : 一个Numpy数组(或类数组), 或者数组的序列(如果模型有多个输入); 一个将名称匹配到对应数组/张量的字典, 如果模型具有命名输入; 一个返回(inputs, targets)或(inputs, targets, sample weights)的生成器或keras.utils.Sequence; None(默认), 如果从本地框架张量馈送(例如TensorFlow数据张量)

**y** : 目标数据。与输入数据x类似, 它可以是Numpy 数组(序列)、本地框架张量(序列)、Numpy数组序列(如果模型有多个输出)或None(默认)如果从本地框架张量馈送(例如TensorFlow数据张量)。如果模型输出层已命名, 也可以传递一个名称匹配Numpy数组的字典。如果x是一个生成器, 或keras.utils.Sequence实例, 则不应该指定y(因为目标可以从x获得)

**batch\_size** : 整数或None。每次梯度更新的样本数。如果未指定, 默认为32。如果你的数据是符号张量、生成器或Sequence实例形式, 不要指定batch\_size, 因为它们会生成批次

**epochs** : 整数。训练模型迭代轮次。一个轮次是在整个x或y上的一轮迭代。请注意, 与initial\_epoch一起, epochs被理解为「最终轮次」。模型并不是训练了epochs轮, 而是到第epochs轮停止训练

**validation\_split** : 0和1之间的浮点数。用作验证集的训练数据的比例。模型将分出一部分不会被训练的验证数据, 并将在每一轮结束时评估这些验证数据的误差和任何其他模型指标。验证数据是混洗之前x和y数据的最后一部分样本中。这个参数在x是生成器或Sequence实例时不支持

**validation\_data** : 用于在每个轮次结束后评估损失和任意指标的数据, 模型不会在这个数据上训练, validation\_data会覆盖validation\_split。validation\_data可以是 : 元组(x\_val, y\_val)或Numpy数组或张量; 元组(x\_val, y\_val, val\_sample\_weights)或Numpy数组; 数据集或数据集迭代器。对于前两种情况, 必须提供batch\_size。对于最后一种情况, 必须提供validation\_steps

**validation\_steps** : 只有在提供了validation\_data并且是一个生成器时才有用, 表示在每个轮次结束时执行验证时, 在停止之前要执行的步骤总数(样本批次)

**shuffle** : 布尔值(是否在每轮迭代之前混洗数据)或者字符串(batch)。batch是处理HDF5数据限制的特殊选项, 它对一个batch内部的数据进行混洗。当steps\_per\_epoch非None时, 这个参数无效

**steps\_per\_epoch** : 整数或None, 在声明一个轮次完成并开始下一个轮次之前的总步数(样品批

次)。使用TensorFlow数据张量等输入张量进行训练时，默认值None等于数据集中样本的数量除以batch的大小，如果无法确定，则为1

```
model.fit(X_train, y_train, epochs=10)
```

训练结果

```
Epoch 1/10
9/9 [=====] - 44s 54ms/step - loss: 5.7171 - accuracy: 0.0135
Epoch 2/10
9/9 [=====] - 0s 18ms/step - loss: 3.6793 - accuracy: 0.1551
Epoch 3/10
9/9 [=====] - 0s 18ms/step - loss: 3.6098 - accuracy: 0.1124
Epoch 4/10
9/9 [=====] - 0s 19ms/step - loss: 3.3768 - accuracy: 0.2298
Epoch 5/10
9/9 [=====] - 0s 19ms/step - loss: 2.7783 - accuracy: 0.5148
Epoch 6/10
9/9 [=====] - 0s 19ms/step - loss: 1.4902 - accuracy: 0.8003
Epoch 7/10
9/9 [=====] - 0s 19ms/step - loss: 0.4353 - accuracy: 0.9280
Epoch 8/10
9/9 [=====] - 0s 19ms/step - loss: 0.1315 - accuracy: 0.9847
Epoch 9/10
9/9 [=====] - 0s 19ms/step - loss: 0.0376 - accuracy: 0.9973
Epoch 10/10
9/9 [=====] - 0s 19ms/step - loss: 0.0135 - accuracy: 1.0000
```

```
<keras.callbacks.History at 0x7f0449705a10>
```

- 通过打印输出结果可以看到，accuracy的评分可以达到很高，损失值也很低

## 步骤7：测试

- 先进行预测

```
y_predict = model.predict(X_test)
```

- 预测后，然后和测试标签进行比对

```
print(y_test[0], np.argmax(y_predict[0]))
```

测试结果

- 预测准确

## 五、实验小结

---

- 基于CNN的人脸识别，从数据准备和处理，训练和测试这几个部分对人脸识别步骤进行实践。训练的人脸识别模型，具有较低的损失以及较高的准确率，测试性能好，说明了模型的有效性